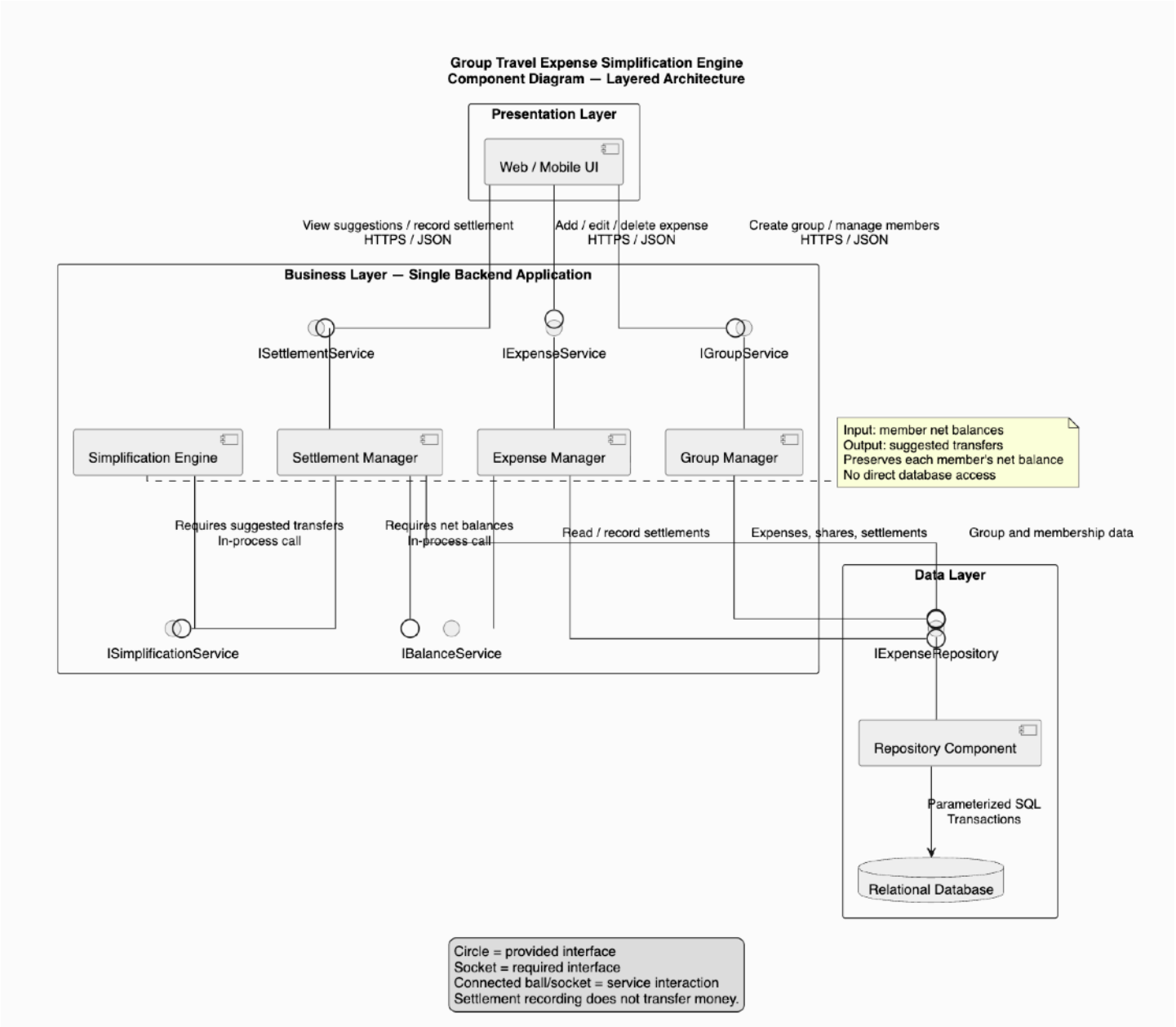
SE Lab 3 - Component diagram and architectural pattern selection

Name: Rohan Rajeev Chirbi
SRN: PES2UG24CS408
Sec: G

Component diagram from PlantUML



Chose layered architecture for the group travel expense simplification engine, organised into Business, Presentation and Data layers. The business components run within a single backend application, rather than as independently deployed micro-services.

Two reasons

1. Clear separation of concerns and easier testing:
The system separates group membership, expense handling, settlement management, and expense simplification into dedicated components. For example, changing the settlement algorithm does not require rewriting the user interface or database access code, provided its interface remains unchanged. The simplification engine accepts member balances and returns suggested transfers, allowing it to be tested independently.

2. Simpler deployment and consistent financial records:
A single backend and relational database avoid the additional deployment and distributed-transaction complexity of micro-services. An expense and its participant shares are saved in one database transaction, preventing partially recorded expenses. Recording settlements transactionally also helps keep the underlying records consistent; appropriate concurrency controls are still necessary.

Security advantage

Centralised authorisation and controlled database access:
The presentation layer cannot directly access the database. Business components authenticate users and verify group membership before allowing access to expenses or settlements. The Repository Component uses parameterised queries, while database credentials remain on the backend. This reduces unauthorised access and SQL-injection risk.

Performance benefit

Low-overhead balance processing and simplification:
The Expense Manager, Settlement Manager, and simplification engine communicate through in-process calls rather than repeated network requests. The system retrieves the necessary expense and settlement records, calculates balances, and generates transfer suggestions in memory. This avoids unnecessary inter-service communication and repeated database access during simplification.